

12주차 과제

TabNet: Attentive Interpretable Tabular Learning

1. 어떤 문제를 해결하려 했는가? (Abstract)

정형 데이터(tabular data)에 대한 고성능 + 해석 가능한 딥러닝 아키텍처가 없다는 문제를 해결하고자 했습니다. 기존에는 XGBoost·LightGBM 같은 트리 앙상블이 정형 데이터를 지배하고 있었으며, 딥러닝은 이 영역에서 경쟁력을 갖추지 못했습니다.

목표	방법
고성능	Sequential Attention 기반 희소 피처 선택
해석 가능성	피처 마스크 시각화 (Local + Global)
준지도 학습	Masked Self-Supervised Pre-training

2. 연구 동기와 문제점 (Introduction)

왜 정형 데이터에 딥러닝이 어려운가?

1. **표현 효율성** — 정형 데이터의 결정 경계는 대부분 초평면(hyperplane) 형태이며, Decision Tree가 이를 잘 포착함
2. **해석 가능성** — 결정 노드 추적이나 SHAP 같은 후처리 설명 기법이 잘 발달되어 있음
3. **빠른 학습** — 트리는 훈련이 빠름

기존 DNN의 문제점

- CNN/MLP는 정형 데이터에 적합한 **귀납적 편향(inductive bias)**이 없어 과매개변수화(over-parameterized)
- 최적 결정 경계를 찾지 못하는 경우가 많음

그럼에도 딥러닝을 써야 하는 이유

- 이미지+정형 데이터를 **end-to-end**로 동시 학습 가능
- Feature engineering 최소화
- 스트리밍 데이터 학습 가능

- 표현 학습(representation learning) → 도메인 적응, 생성 모델, 준지도 학습 가능

3. 관련 연구 동향 (Related Works)

카테고리	주요 방법	한계
전역 피처 선택	Forward Selection, Lasso	인스턴스별 선택 불가
인스턴스별 피처 선택	L2X (Chen et al. 2018)	별도 설명 모델 필요
인스턴스별 피처 선택	INVASE (Yoon et al. 2019)	Actor-Critic 구조로 모델 3개 필요 → 파라미터 비효율
트리 기반 앙상블	XGBoost, LightGBM, CatBoost	DNN과의 end-to-end 통합 불가
DNN+DT 통합	Soft DT, Neural DT	자동 피처 선택 기능 손실
Self-Supervised	BERT (text), Selfie (image)	정형 데이터에 적용한 선례 없음

TabNet의 차별점: 단일 아키텍처에서 피처 선택 + 출력 매핑을 동시에 수행하며, 최초로 정형 데이터에 자기지도 사전 학습을 적용

4. 연구 접근법과 모델 구조 (Method)

전체 구조 개요

```

입력 피처  $f \in \mathbb{R}^{(B \times D)}$ 
  ↓ Batch Normalization
[Step 1] Attentive Transformer → Mask  $M[1]$  → Feature Transformer →  $d[1], a[1]$ 
[Step 2] Attentive Transformer → Mask  $M[2]$  → Feature Transformer →  $d[2], a[2]$ 
  ...
[Step N] Attentive Transformer → Mask  $M[N]$  → Feature Transformer →  $d[N], a[N]$ 
  ↓
최종 출력 =  $W_{final} \times \sum \text{ReLU}(d[i])$ 

```

핵심 구성 요소 4가지

① Attentive Transformer (피처 선택)

각 스텝에서 어떤 피처를 볼지 결정하는 소프트 마스크를 생성합니다.

- **sparsemax**: softmax 대신 사용 — 유클리드 투영 기반으로 희소성을 자연스럽게 유도 (많은 값이 정확히 0이 됨)

② Prior Scale (재사용 제어)

이미 사용된 피처를 얼마나 다시 쓸지 제어합니다.

③ Sparsity Regularization

마스크의 희소성을 엔트로피로 추가 제어합니다.

④ Feature Transformer (피처 처리)

선택된 피처를 비선형 변환합니다.

마스킹된 피처 $M[i] \cdot f$

- [공유 FC → BN → GLU] × 2 ← 모든 스텝 공유
- [스텝별 FC → BN → GLU] × 2 ← 각 스텝 독립
- residual connection ($\sqrt{0.5}$ 스케일)
- split → $d[i]$ (출력), $a[i]$ (다음 스텝 전달)

GLU (Gated Linear Unit): 게이팅으로 불필요한 정보를 억제

해석 가능성 계산

Local (인스턴스별) 피처 중요도

Self-Supervised Pre-training (자기지도 사전 학습)

BERT의 마스킹 언어 모델 방식을 정형 데이터에 적용합니다.

이진 마스크 $S \in \{0, 1\}^{(B \times D)}$ (Bernoulli 샘플링)

Encoder 입력: $(1 - S) \cdot \hat{f}$ (관측된 피처만 입력)

Decoder 출력: $S \cdot \hat{f}$ (가려진 피처를 예측)

재구성 손실:

$$\sum_b \sum_j |(\hat{f}_{bj} - f_{bj}) \cdot S_{bj} / \text{std}(f_j)|^2$$

5. 실험 구성과 결과 (Experiments)

실험 데이터셋

데이터셋	태스크	규모
Syn1~Syn6	피처 선택 검증 (합성)	10K
Forest Cover Type	분류	581K
Poker Hand	분류	1M
Sarcos	회귀	44K
Higgs Boson	분류	10.5M
Rossmann Store Sales	회귀	대용량
KDD (Appetency/Churn 등)	분류	대용량

주요 결과

Forest Cover Type 분류 정확도

모델	정확도
XGBoost	89.34%
LightGBM	89.28%
AutoML Tables	94.95%
TabNet	96.99%

Poker Hand 분류 정확도

모델	정확도
DT / MLP	50.0%
XGBoost	71.1%
TabNet	99.2%

Sarcos 회귀 MSE

모델	MSE
Gradient Boosted Tree	1.44
TabNet-M	0.28
TabNet-L	0.14

Self-Supervised Pre-training 효과 (Higgs, 1K 학습 데이터)

방식	정확도
Supervised only	57.47 ± 1.78%
With pre-training	61.37 ± 0.88%

→ 레이블 데이터가 적을수록 사전 학습 효과가 극적으로 증가

6. 무엇을 발견했고 한계점은? (Discussion)

주요 발견

- **Instance-wise 피쳐 선택**이 Global 피쳐 선택보다 우수 (Syn4~6 결과)
- Sparsemax가 softmax보다 해석 가능성과 성능 모두에서 우수
- **대형 배치 학습**이 TabNet에서 특히 중요 (Ghost BN 덕분에 가능)
- GLU가 ReLU보다 일관되게 성능 우수
- 파라미터 **부분 공유** (일부 공유 + 일부 스텝 독립)가 최적

Ablation Study 핵심 결과

변경 내용	정확도 변화
기본 모델	96.99%
Nsteps 3으로 감소	-0.77%
단일 공유 레이어만 사용	-1.67%
GLU → ReLU 교체	-3.07%
$\lambda_{\text{sparse}} = 0.01$ (과도한 희소성)	-3.12%

한계점

- KDD 데이터셋처럼 포화(saturated) 벤치마크에서는 XGBoost·CatBoost 대비 미소한 우위 또는 동등
- 하이퍼파라미터(Nsteps, γ , λ_{sparse} , 배치 크기)가 많아 튜닝 부담 존재
- 특수 설계된 피쳐 엔지니어링(로그 변환 등)은 논문 범위에서 제외 → 실무 적용 시 추가 고려 필요
- 매우 작은 데이터셋에서는 트리 모델이 여전히 유리할 수 있음

7. 결론 및 주요 요약 (Conclusion)

1. TabNet은 전처리 없이 원시 정형 데이터를 입력받아 gradient descent로 end-to-end 학습 가능
2. Sequential Attention 기반 인스턴스별 피처 선택으로 해석 가능성과 학습 효율을 동시에 달성
3. 다양한 도메인 벤치마크에서 XGBoost·LightGBM·CatBoost 등 트리 앙상블을 능가
4. 정형 데이터에서 최초로 자기지도 사전 학습을 적용하여 레이블 부족 상황에서 유의미한 성능 향상 달성

8. 기존 연구 대비 차별점

비교 대상	TabNet의 차별점
XGBoost / LightGBM	딥러닝이므로 end-to-end, 멀티모달, 표현학습 가능
L2X	별도 설명 모델 없이 단일 모델 에서 선택+추론 통합
INVASE	Actor-Critic 3모델 구조 대신 단일 컴팩트 아키텍처
Soft/Neural DT	자동 피처 선택 기능을 유지하면서 DNN 결합
BERT	텍스트가 아닌 정형 데이터 에 마스킹 자기지도 학습 최초 적용
기존 MLP	구조화된 희소성(structured sparsity) → 연산 효율 저하 없음

9. 핵심 아이디어 요약

|"결정 트리처럼 피처를 골라서, 신경망처럼 처리하자"

Decision Tree의 장점: 피처 선택, 해석 가능성
DNN의 장점: 비선형 표현력, end-to-end, 준지도 학습

↓

TabNet = Sequential Attention으로 스텝마다 중요 피처만 선택
+ Feature Transformer로 선택된 피처를 깊게 처리
+ 마스크를 그대로 해석 도구로 활용

핵심 인사이트는 **sparsemax**를 이용한 희소 소프트 마스킹입니다. softmax는 모든 값이 0보다 크지만, sparsemax는 많은 값을 정확히 0으로 만들어 — 실질적으로 "이 피처는 이번 스텝에서 무시"를 학습 가능하게 합니다.

10. 주요 수식과 코드 분석

핵심 수식 정리

```
# 1. 피쳐 마스크 생성
M[i] = sparsemax(P[i-1] · h_i(a[i-1]))

# 2. Prior Scale 업데이트 (재사용 억제)
P[i] = ∏_{j=1}^i (γ - M[j])

# 3. 피쳐 처리 (마스킹 후 변환)
[d[i], a[i]] = f_i(M[i] · f)

# 4. 최종 출력 집계
d_out = ∑_{i=1}^{N_steps} ReLU(d[i])
output = W_final · d_out

# 5. 희소성 정규화 손실
L_sparse = ∑_i ∑_b ∑_j (-M_bj[i] · log(M_bj[i] + ε)) / (N_steps · B)
```

PyTorch 스타일 구조 코드

```
import torch
import torch.nn as nn
from torch.nn import functional as F

# -----
# Sparsemax: softmax 대신 사용하여 희소 확률 분포 생성
# 유클리드 프로젝션 기반으로 많은 값이 정확히 0이 됨
# -----
class Sparsemax(nn.Module):
    def forward(self, z):
        # z: (B, D) 형태의 입력
        z_sorted, _ = torch.sort(z, descending=True, dim=-1)

        # 누적합으로 threshold 계산
        cumsum = torch.cumsum(z_sorted, dim=-1)
        k = torch.arange(1, z.shape[-1] + 1, device=z.device).float()
```

```

        condition = 1 + k * z_sorted > cumsum
        # 마지막으로 조건 만족하는 인덱스 = k_hat
        k_hat = condition.sum(dim=-1, keepdim=True).float()
        tau = (cumsum.gather(1, (k_hat - 1).long()) - 1) /
k_hat

        # ReLU로 0 이하를 정확히 0으로 클리핑
        return torch.clamp(z - tau, min=0)

# -----
# GLU (Gated Linear Unit): 게이팅으로 정보 흐름 제어
# -----
class GLU(nn.Module):
    def forward(self, x):
        # 입력을 반으로 나눠 하나는 값, 하나는 게이트
        x1, x2 = x.chunk(2, dim=-1)
        return x1 * torch.sigmoid(x2)

# -----
# Feature Transformer: 공유 레이어 + 스텝별 레이어
# -----
class FeatureTransformer(nn.Module):
    def __init__(self, input_dim, output_dim, shared_fc, step_fc):
        super().__init__()
        # 모든 스텝에서 공유되는 FC 레이어 (외부에서 주입)
        self.shared_fc = shared_fc
        self.shared_bn = nn.BatchNorm1d(output_dim * 2)

        # 각 스텝에 독립적인 FC 레이어
        self.step_fc = step_fc
        self.step_bn = nn.BatchNorm1d(output_dim * 2)

        self.glu = GLU()
        self.scale = (0.5 ** 0.5) # 잔차 연결 분산 안정화

    def forward(self, x):

```

```

# 공유 레이어 통과
h = self.glu(self.shared_bn(self.shared_fc(x)))
# 잔차 연결 ( $\sqrt{0.5}$  스케일)
h = self.scale * (h + x[:, :h.shape[-1]])

# 스텝별 레이어 통과
h = self.glu(self.step_bn(self.step_fc(h)))
h = self.scale * (h + x[:, :h.shape[-1]])
return h

# -----
# TabNet Encoder: 전체 Sequential Attention 구조
# -----
class TabNetEncoder(nn.Module):
    def __init__(self, input_dim, output_dim, n_steps=5,
                  n_d=64, n_a=64, gamma=1.5, lambda_sparse=1
e-4):
        super().__init__()
        self.n_steps = n_steps
        self.gamma = gamma # Prior scale
완화 파라미터
        self.lambda_sparse = lambda_sparse # 희소성 정규화
강도

        # 입력 BN (ghost BN 아닌 일반 BN 사용)
        self.input_bn = nn.BatchNorm1d(input_dim)

        # 공유 FC (모든 스텝에서 동일하게 사용)
        self.shared_fc = nn.Linear(input_dim, n_d * 2, bias
=False)

        # 각 스텝의 Attentive Transformer
        self.attention_transformers = nn.ModuleList([
            nn.Linear(n_a, input_dim, bias=False)
            for _ in range(n_steps)
        ])
        self.attention_bns = nn.ModuleList([

```

```

        nn.BatchNorm1d(input_dim) for _ in range(n_steps)
s)

    ])

# 각 스텝의 Feature Transformer (공유 레이어 주입)
self.feature_transformers = nn.ModuleList([
    FeatureTransformer(
        input_dim, n_d,
        shared_fc=self.shared_fc,
        step_fc=nn.Linear(n_d, n_d * 2, bias=False)
    ) for _ in range(n_steps)
])

self.sparsemax = Sparsemax()
self.final_fc = nn.Linear(n_d, output_dim)

def forward(self, x):
    # 입력 배치 정규화
    x = self.input_bn(x)
    B, D = x.shape

    # Prior scale 초기화: 모든 피쳐 동등 사용 가능
    P = torch.ones(B, D, device=x.device)

    # 이전 스텝 출력 (첫 스텝은 0으로 초기화)
    a_prev = torch.zeros(B, self.feature_transformers
[0].step_fc.out_features // 2,
                        device=x.device)

    total_entropy = 0      # 희소성 정규화 누적
    step_outputs = []      # 각 스텝 출력 수집

    for i in range(self.n_steps):
        # — Attentive Transformer —————

        # 이전 스텝 정보로 현재 스텝 마스크 계산
        h = self.attention_bns[i](self.attention_transf
ormers[i](a_prev))

```

```

# Prior scale 곱하여 이미 많이 쓴 피처를 억제
M = self.sparsemax(P * h)

# Prior scale 업데이트 (γ에서 마스크를 빼는 구조)
P = P * (self.gamma - M)

# 희소성 엔트로피 누적
total_entropy += (-M * torch.log(M + 1e-15)).sum(
dim=-1).mean()

# — Feature Transformer —————
—

# 마스크된 피처를 비선형 변환
masked_x = M * x
h_out = self.feature_transformers[i](masked_x)

# 출력 분리: d는 최종 결합, a는 다음 스텝 전달
n_d = h_out.shape[-1] // 2
d_i, a_prev = h_out[:, :n_d], h_out[:, n_d:]
step_outputs.append(F.relu(d_i))

# 모든 스텝 출력 합산 → 최종 결정 임베딩
d_out = torch.stack(step_outputs, dim=0).sum(dim=0)
output = self.final_fc(d_out)

# 희소성 정규화 손실 반환 (전체 손실에 λ_sparse 곱해 더
함)
sparse_loss = self.lambda_sparse * total_entropy /
self.n_steps
return output, sparse_loss

```

TabPFN v2: Accurate Predictions on Small Data with a Tabular Foundation Model

1. 어떤 문제를 해결하려 했는가? (Abstract)

소규모~중규모 정형 데이터에서 기존 최강자인 **Gradient Boosted Decision Tree(GBDT)**를 단일 **forward pass**로 능가하는 파운데이션 모델을 제시합니다.

TabPFN은 최대 10,000개 샘플을 가진 데이터셋에서 이전 모든 방법을 큰 차이로 능가하며, 훨씬 적은 학습 시간을 사용합니다. 분류 설정에서 2.8초 만에 4시간 동안 튜닝된 가장 강력한 기준선 앙상블을 능가합니다.

목표	달성 방법
소규모 데이터 고성능	In-Context Learning (ICL) 기반 단일 forward pass
데이터셋 간 전이	수백만 개 합성 데이터로 사전 학습
파운데이션 모델 기능	Fine-tuning, 데이터 생성, 밀도 추정, 임베딩 재사용

2. 연구 동기와 문제점 (Introduction)

왜 정형 데이터에 딥러닝이 어려운가?

정형 데이터는 이종성(heterogeneity) 두 가지 문제를 갖고 있습니다. 데이터셋 간 이종성(drug discovery vs. materials science)과 데이터 자체의 이종성(Boolean, categorical, ordinal, integer, float, 결측값, 이상치 등). 이로 인해 트리 기반 모델이 지금까지 가장 강력한 방법이었습니다.

OpenML.org의 인기 있는 정형 벤치마킹 사이트에서 76%의 데이터셋이 10,000개 미만의 행을 포함합니다. → 소규모 데이터 문제가 현실에서 압도적으로 많음

기존 트리 모델의 한계

전통적인 머신러닝 모델들은 분포 외(out-of-distribution) 예측이 취약하고, 한 데이터셋에서 다른 데이터셋으로의 지식 전이가 어려우며, 신경망과의 결합(그래디언트 전파)이 어렵습니다.

딥러닝 역사의 관점

AI 역사 전반에 걸쳐 수동으로 만든 알고리즘 구성 요소들이 end-to-end로 학습된 더 나은 구성 요소로 대체되어 왔습니다. 컴퓨터 비전에서는 SIFT·HOG가 학습된 합성곱으로, NLP에서는 문법 기반 접근이 트랜스포머로 대체되었습니다. TabPFN은 이 흐름을 정형 데이터 영역으로 확장합니다.

3. 관련 연구 동향 (Related Works)

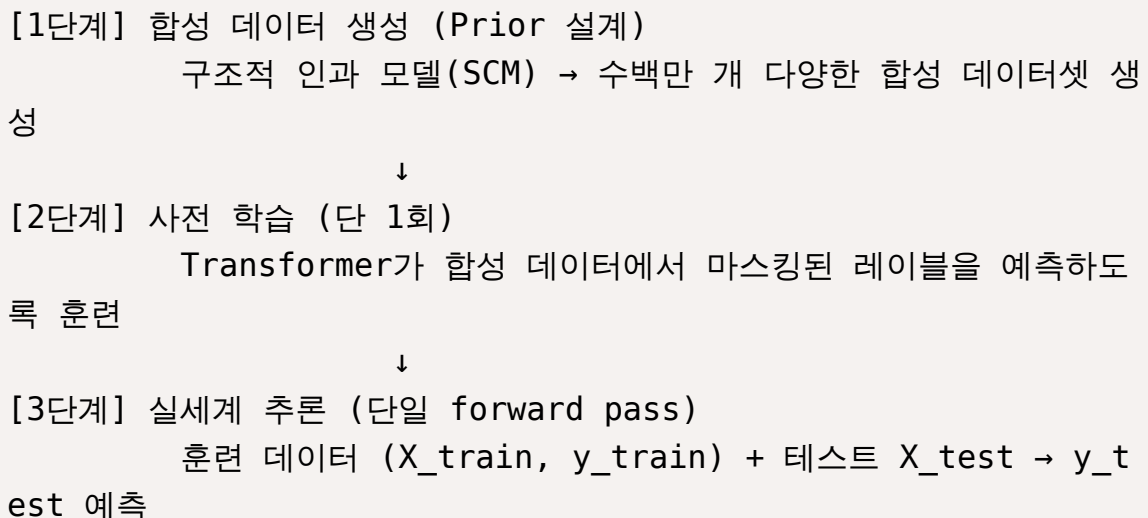
카테고리	주요 방법	한계
GBDT	XGBoost, LightGBM, CatBoost	전이 학습 불가, gradient 미전파
정형 딥러닝	TabNet, FT-Transformer, SAINT	여전히 GBDT에 열세 (소규모 데이터)
ICL 이론	GPT-3, PFN v1	정형 데이터 규모 한계 (1K 샘플, 100 피처)
AutoML	AutoML Tables, AutoGluon	수 시간 튜닝 필요
크로스-테이블 Pre-training	XTab	실제 데이터 프라이버시·오염 문제

PFN(Prior-data Fitted Networks)은 트랜스포머가 ICL을 통해 가우시안 프로세스와 베이지안 신경망 같은 복잡한 알고리즘조차 근사할 수 있음을 보였습니다.

TabPFN v1과의 차이: 전작(ICLR 2023)은 1,000 샘플, 100 피처, 분류만 지원했으며 실용성이 제한적이었음. TabPFN v2는 이를 50배 확장하고 회귀, 범주형 변수, 결측값 처리를 모두 추가

4. 연구 접근법과 모델 구조 (Method)

전체 파이프라인 3단계



핵심 아이디어: In-Context Learning (ICL)

기존 방법과의 근본적 차이:

[기존 딥러닝]

데이터셋 A → 모델 학습 → 테스트 예측

데이터셋 B → 모델 재학습 → 테스트 예측 (매번 새로 학습)

[TabPFN ICL]

사전학습: 합성 데이터셋 수백만 개 → PFN 1회 학습

추론: (X_train, y_train, X_test) → 단일 forward pass → y_test

(모델 파라미터 업데이트 없음. 데이터 자체가 컨텍스트)

기존 접근법은 개별 샘플이나 배치에서 Adam 같은 수동 제작 가중치 갱신 알고리즘으로 모델을 학습시킵니다. 반면 TabPFN은 데이터셋 전체를 추론 시간에 받아, 단일 신경망 forward pass에서 학습과 예측을 동시에 수행합니다.

아키텍처: 2방향 어텐션 (Two-Way Attention)

TabPFN의 아키텍처는 테이블의 각 셀에 별도의 표현을 할당하는 2방향 어텐션 메커니즘을 사용합니다. 각 셀은 자신의 행(동일 샘플의 다른 피쳐들)과 자신의 열(동일 피쳐의 다른 샘플들) 양쪽으로 어텐션을 수행합니다. 이 설계는 아키텍처가 샘플과 피쳐의 순서에 불변하도록 하고, 훈련 시 접한 것보다 더 큰 테이블로의 효율적인 외삽을 가능하게 합니다.

일반 Transformer: 행(샘플)만 시퀀스로 처리

TabPFN Transformer: 행 어텐션 (같은 샘플 내 피쳐 간)

+ 열 어텐션 (같은 피쳐 간 샘플 비교)

→ 순서 불변(permutation invariant) for both samples & features

Train-State Caching (추론 최적화)

훈련 세트에서 한 번 ICL을 수행하고 결과 상태를 저장하여 여러 테스트 세트 추론에 재사용할 수 있습니다. 10,000개 훈련 샘플과 10개 피쳐 데이터셋에서 CPU 기준 약 300배(32초 → 0.1초), GPU 기준 6배의 추론 속도 향상을 달성합니다. 피쳐가 100개로 늘어나면 CPU 800배, GPU 30배 향상됩니다.

합성 데이터 생성: 구조적 인과 모델 (SCM)

합성 데이터를 실제 데이터 대신 사용함으로써 파운데이션 모델의 일반적인 문제들, 즉 프라이버시 침해, 저작권 문제, 테스트 데이터로의 훈련 데이터 오염을 방지합니다.

- ① 고수준 하이퍼파라미터 샘플링
(데이터셋 크기, 피쳐 수, 난이도 등)
↓
- ② DAG (방향성 비순환 그래프) 구성
→ 인과 구조 정의
↓
- ③ 초기화 데이터 (노이즈) → 루트 노드 전파
각 엣지에 적용되는 변환:
 - 소형 신경망 (선형/ReLU/sigmoid/sin)
 - 이산화 메커니즘 (범주형 피쳐)
 - 결정 트리 구조 (규칙 기반 의존성)
 - 각 엣지마다 Gaussian noise 추가
 ↓
- ④ 각 노드의 중간 표현 저장 → 피쳐/타겟 노드에서 추출
↓
- ⑤ 후처리 (Kumaraswamy 분포 워핑, 양자화 등)
↓
- ⑥ ~1억 개 합성 데이터셋 생성 완료

회귀를 위한 분포 출력

회귀 태스크에서 구간별 상수 출력 분포(piece-wise constant output distribution)를 사용하여, 단일 값이 아닌 타겟 값의 확률 분포를 예측할 수 있습니다. 이중 봉우리(bimodal) 분포도 포함됩니다.

5. 실험 구성과 결과 (Experiments)

벤치마크

- **AutoML Benchmark:** 분류 29개 데이터셋
- **OpenML-CTR23:** 회귀 36개 데이터셋
- 비교 대상: XGBoost, LightGBM, CatBoost (4시간 튜닝), AutoGluon, TabNet, FT-Transformer 등

핵심 결과

속도 비교

방법	소요 시간
GBDT 앙상블 (4시간 튜닝)	~14,400초
TabPFN (분류)	2.8초
TabPFN (회귀)	단일 forward pass

→ 분류 설정에서 5,140배, 회귀 설정에서 3,000배의 속도 향상을 달성합니다.

질적 분석 (토이 함수)

이중 슬릿 실험 (복잡한 분포 예측)

TabPFN은 이중 슬릿 실험의 복잡한 다봉(multi-modal) 강도 패턴을 단 1.2초에 단일 forward pass로 예측합니다. 기본 설정 CatBoost는 169.3초가 필요하며 훨씬 나쁜 결과를 냅니다.

파운데이션 모델 기능 실증

- **Fine-tuning:** 특정 도메인 데이터로 사전 학습 모델을 추가 학습
- **데이터 생성:** 학습된 분포에서 합성 샘플 생성
- **밀도 추정:** 결합 확률 분포 추정
- **임베딩 재사용:** 다운스트림 태스크용 표현 추출

6. 무엇을 발견했고 한계점은? (Discussion)

주요 발견

1. **알고리즘을 학습하는 알고리즘:** TabPFN은 단일 예측 모델이 아니라, 임의의 정형 데이터 문제를 푸는 **학습 알고리즘 자체를 학습함**
2. **불확실성 추정 내재화:** 추가 구현 없이 예측 분포를 반환 — 기존 방법은 앙상블이나 분위수 회귀 필요
3. **결측값·이상치 자동 처리:** 합성 데이터에 이러한 도전 과제들을 포함시켜 학습함으로써 실세계 문제를 자동 처리
4. **베이지안 해석:** TabPFN은 근사 베이지안 추론으로 해석될 수 있으며, 합성 데이터로 정의된 Prior에 대한 사후 예측 분포를 근사합니다.

한계점

- **10,000 샘플 이상의 대용량 데이터에서는 성능 이점 감소** — 포화된 벤치마크에서 GBDT가 경쟁적

- 피쳐 수 500개 초과 시 성능 저하 (v2 기준. v2.5에서 2,000개로 확장)
- GPU 의존적: CPU에서는 소규모($\leq 1,000$ 샘플)만 실용적
- 해석 가능성 부재: TabNet과 달리 native interpretability가 없음
- 메모리 요구사항 최적화 후에도 H100 GPU에서 최대 5천만 셀(예: 5백만 행 $\times$ 10 피처)로 제한됩니다.

7. 결론 및 주요 요약 (Conclusion)

1. TabPFN은 정형 데이터를 위한 최초의 실용적 파운데이션 모델로, 수백만 개의 합성 데이터셋으로 학습된 알고리즘
2. ICL을 통해 추론 시 파라미터 업데이트 없이 단일 forward pass로 예측
3. 소규모 데이터($\leq 10K$)에서 4시간 튜닝된 GBDT 앙상블을 2.8초 만에 능가 (5,140배 빠름)
4. 분류+회귀 모두 지원하며, 불확실성 추정·Fine-tuning·데이터 생성·임베딩 재사용 등 파운데이션 모델 기능 보유
5. 합성 데이터 사용으로 프라이버시·오염·저작권 문제 회피

8. 기존 연구 대비 차별점

비교 대상	TabPFN의 차별점
XGBoost / LightGBM / CatBoost	데이터셋별 재학습 불필요. 2.8초 vs 4시간
TabNet	해석 가능성 대신 성능과 속도 압도, 파운데이션 모델 기능
XTab / 기타 크로스-테이블 Pre-training	실제 데이터가 아닌 합성 데이터 사용 → 오염·프라이버시 문제 없음
PFN v1 (ICLR 2023)	50배 큰 데이터 처리, 회귀 지원, 범주형·결측값 처리 추가
BERT/GPT 스타일	텍스트의 토큰 대신 테이블 셀 단위 2방향 어텐션으로 정형 구조 활용
AutoML (AutoGluon)	앙상블·탐색 불필요. 단일 모델로 동등 혹은 우월 성능

9. 핵심 아이디어

|"데이터를 학습하는 대신, 학습하는 방법 자체를 학습한다"

전통적 ML: 데이터 → 알고리즘(사람이 설계) → 모델
 TabPFN: 합성 데이터 수백만 개 → 트랜스포머가 알고리즘 자체를 학습

↓

 새 데이터 도착 → 학습된 알고리즘이 즉시 적용 (forward pass)

세 가지 핵심 인사이트:

1. **SCM 기반 Prior** — 실제 데이터 없이도 현실적인 정형 데이터의 다양성을 포착하는 생성 프로세스 설계
2. **2방향 어텐션** — 행(샘플)과 열(피처) 양방향으로 어텐션을 수행하여 테이블 구조를 자연스럽게 처리
3. **ICL as Algorithm** — 수백만 개 과제에서 학습한 트랜스포머가 추론 시 컨텍스트만으로 새 과제를 즉시 풀 수 있음

10. 주요 수식과 코드 분석

베이지안 사후 예측 분포 근사

→ 테스트 레이블의 분포 전체를 반환 (단일 값이 아님)

회귀 손실 (구간별 상수 분포)

→ 연속 타겟을 이산 bin으로 처리하여 분류처럼 학습

사용 코드 (실제 API 기준)

```
# _____
#
# TabPFN v2 설치 및 기본 사용법
# pip install tabpfn
# _____

from tabpfn import TabPFNClassifier, TabPFNRegressor
from sklearn.datasets import make_classification, make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_er
```

```

ror
import numpy as np

# — 분류 예시 —————
# 합성 이진 분류 데이터 생성
X, y = make_classification(
    n_samples=500,          # 소규모 데이터에서 강점 발휘
    n_features=20,
    n_informative=10,
    random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# TabPFN은 전처리 불필요 (스케일링, 원핫인코딩 직접 처리)
clf = TabPFNClassifier()

# fit: 데이터를 컨텍스트로 저장 (파라미터 업데이트 없음)
clf.fit(X_train, y_train)

# predict: 단일 forward pass로 추론
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test) # 불확실성 포함 확률 분포

print(f"Classification Accuracy: {accuracy_score(y_test, y_
pred):.4f}")

# — 회귀 예시 —————
X_r, y_r = make_regression(n_samples=500, n_features=15, ra
ndom_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X_r, y_r, test_si
ze=0.2)

reg = TabPFNRegressor()

```

```

reg.fit(X_tr, y_tr)

y_pred_r = reg.predict(X_te)
print(f"Regression MSE: {mean_squared_error(y_te, y_pred_r):.4f}")

# — 실전 팁 —————
# (1) 배치 예측: 테스트를 한 번에 predict - 개별 호출은 100배 느림
y_pred_batch = clf.predict(X_test)          # 올바른 방법
# y_preds = [clf.predict(x.reshape(1,-1)) for x in X_test]
# 느림!

# (2) 대규모 테스트셋은 청크 분할
chunk_size = 1000
preds = []
for i in range(0, len(X_test), chunk_size):
    chunk = X_test[i:i+chunk_size]
    preds.append(clf.predict(chunk))
y_pred_chunked = np.concatenate(preds)

# — 아키텍처 내부 개념 코드 (Two-Way Attention 의사 코드) ———
import torch
import torch.nn as nn

class TwoWayAttention(nn.Module):
    """
    TabPFN의 핵심: 행(샘플)과 열(피처) 방향 어텐션을 순차 적용
    입력: (B, N_samples, N_features, D_hidden)
    """
    def __init__(self, d_model, n_heads):
        super().__init__()
        # 행 방향 어텐션: 같은 샘플 내 피처들 간 관계 학습
        self.row_attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)

```

```

        # 열 방향 어텐션: 같은 피쳐 위치의 샘플들 간 관계 학습
        self.col_attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, x):
        # x shape: (B * N_samples, N_features, D) → 행 어텐션
        B, N, F, D = x.shape

        # 1. 행 어텐션: 각 샘플 내 피쳐들 간 상호작용
        x_row = x.view(B * N, F, D)
        x_row, _ = self.row_attn(x_row, x_row, x_row)
        x = self.norm1(x + x_row.view(B, N, F, D))

        # 2. 열 어텐션: 같은 피쳐 기준 샘플들 간 상호작용
        # → 훈련 샘플이 테스트 샘플에 영향을 미치는 핵심 메커니즘
        x_col = x.permute(0, 2, 1, 3).reshape(B * F, N, D)
        x_col, _ = self.col_attn(x_col, x_col, x_col)
        x = self.norm2(x + x_col.view(B, F, N, D).permute(0, 2, 1, 3))

        return x

# — 합성 데이터 Prior의 개념적 구현 —
import numpy as np
import networkx as nx

def generate_synthetic_dataset(n_samples=100, n_features=10, seed=None):
    """
    TabPFN Prior의 SCM 기반 합성 데이터 생성 (간소화 버전)
    실제 구현은 훨씬 복잡한 하이퍼파라미터와 변환 함수를 사용
    """
    rng = np.random.RandomState(seed)

```

```

# 1. 랜덤 DAG 생성 (인과 구조)
n_nodes = n_features + 2 # 피쳐 노드 + 타겟 노드
adjacency = np.tril(rng.binomial(1, 0.3, (n_nodes, n_nodes)), -1)

# 2. 루트 노드에 노이즈 주입 → 엣지 변환 함수 적용
node_values = {}
activation_fns = [np.tanh, np.sin, lambda x: np.maximum(0, x)] # 다양한 활성화

for node in range(n_nodes):
    parents = np.where(adjacency[node] == 1)[0]
    if len(parents) == 0:
        # 루트 노드: 순수 노이즈
        node_values[node] = rng.randn(n_samples)
    else:
        # 부모 노드들의 가중 합 + 변환 + 노이즈
        W = rng.randn(len(parents))
        h = sum(W[i] * node_values[p] for i, p in enumerate(parents))
        fn = activation_fns[rng.randint(len(activation_fns))]
        node_values[node] = fn(h) + 0.1 * rng.randn(n_samples)

# 3. 피쳐와 타겟 노드 위치 샘플링
feature_nodes = rng.choice(n_nodes - 1, n_features, replace=False)
target_node = n_nodes - 1

X = np.column_stack([node_values[n] for n in feature_nodes])
y = node_values[target_node]

return X, y

# 사용 예시

```

```
X_syn, y_syn = generate_synthetic_dataset(n_samples=200, n_
features=8, seed=42)
print(f"합성 데이터 생성 완료: X={X_syn.shape}, y={y_syn.shap
e}")
```